CHAPTER 5 – PYTHON-BASED LAND COVER CLASSIFICATION USING RANDOM FOREST

5.1 Introduction

The previous chapter presented the complete land-cover classification workflow implemented using QGIS and the Semi-Automatic Classification Plugin (SCP). The workflow included satellite image preprocessing, generation of True Color Composite and False Color Composite images, Normalized Difference Vegetation Index (NDVI) analysis, supervised and unsupervised classification, and accuracy assessment. Among the evaluated classification techniques, the Random Forest classifier produced the highest classification accuracy and generated the most reliable land-cover map for the selected study area.

Although QGIS provides an efficient graphical environment for satellite image processing and classification, modern remote sensing applications increasingly require automated, reproducible, and customizable workflows. Programming languages such as Python provide greater flexibility for handling large geospatial datasets, integrating machine learning algorithms, automating repetitive tasks, and developing scalable image processing pipelines. Python also offers a wide range of open-source libraries for raster processing, vector data handling, scientific computing, visualization, and machine learning, making it one of the most widely used programming languages in remote sensing and geospatial analysis.

Based on the results obtained in Chapter 4, the Random Forest algorithm was selected for the Python implementation because it achieved the best classification performance among the evaluated supervised and unsupervised classification methods. Rather than re-implementing all classification techniques, the Python workflow focuses on reproducing the complete Random Forest–based land-cover classification process in a fully automated manner. This implementation demonstrates how satellite image preprocessing, feature extraction, machine learning model training, classification, visualization, and accuracy assessment can be performed programmatically without relying on graphical user interfaces.

The Python implementation was developed using Jupyter Notebook and several open-source libraries, including NumPy, Pandas, Rasterio, GeoPandas, Matplotlib, Scikit-learn, and Joblib. These libraries were used to process Landsat 9 multispectral imagery, generate RGB and FCC composites, compute the NDVI, prepare training and validation datasets, train the Random Forest classifier, classify the study area, evaluate classification accuracy, and export the final outputs.

The workflow implemented in this chapter follows a systematic sequence beginning with loading and preparing the Landsat 9 subset imagery, followed by multispectral image visualization, vegetation analysis, training sample preparation, Random Forest model training, land-cover classification, and quantitative accuracy assessment using independent validation data. The generated outputs include classified raster maps, visualization products, trained machine learning models, confusion matrices, classification reports, and accuracy assessment tables.

The primary objective of this chapter is to demonstrate a complete Python-based implementation of land-cover classification using the Random Forest algorithm and to compare its performance and workflow with the QGIS-based implementation presented in the previous chapter. The chapter highlights the advantages of programmatic geospatial analysis, emphasizing reproducibility, automation, and flexibility while maintaining comparable classification accuracy for the Landsat 9 study area.

5.2 Development Environment

The Python-based land-cover classification workflow was developed using an integrated open-source software environment designed for geospatial data processing, scientific computing, machine learning, and data visualization. Unlike the graphical workflow implemented in QGIS, the Python implementation provides a programmatic approach that enables automation, reproducibility, and greater flexibility in processing Landsat 9 multispectral imagery.

The development environment was configured to perform all stages of the workflow, including satellite image loading, preprocessing, RGB and False Color Composite generation, NDVI computation, preparation of training and validation datasets, Random Forest model training, land-cover classification, visualization, and accuracy assessment. Jupyter Notebook was used as the primary development platform, allowing source code, outputs, figures, and documentation to be organized within a single interactive environment.

The software applications and Python libraries used throughout the implementation are described in the following sections.

5.2.1 Software Requirements

The Python implementation was developed using **Python** in the **Jupyter Notebook** environment. Jupyter Notebook provides an interactive platform for writing, executing, and documenting Python programs while simultaneously displaying outputs, figures, and explanatory text. The source code was developed and managed using Visual Studio Code, whereas Git and GitHub were used for version control and project management throughout the development process.

The software tools used in this study are listed in Table 5.1.

Table 5.1: Software Used for Python Implementation

Software	Purpose
Python	Programming language used for implementing the complete land-cover classification workflow
Jupyter Notebook	Interactive environment for developing, executing, and documenting the workflow
Visual Studio Code	Source code editing and project management
Git	Version control and source code management
GitHub	Repository hosting and project version management

5.2.2 Python Libraries Used

Several open-source Python libraries were used to implement different stages of the land-cover classification workflow. These libraries provide specialized functionality for numerical computation, geospatial data processing, visualization, machine learning, and model persistence. Their combined use enabled the complete satellite image processing and classification workflow to be implemented programmatically without relying on proprietary software.

The Python libraries used in this study are summarized in Table 5.2.

Table 5.2: Python Libraries Used in the Implementation

Library	Purpose
NumPy	Numerical computation, array manipulation, and mathematical operations on raster data
Pandas	Data manipulation, tabular data handling, and exporting classification and accuracy assessment results to CSV files
Rasterio	Reading, writing, and processing raster datasets
GeoPandas	Reading and processing vector datasets such as training and validation ROIs
Matplotlib	Visualization of satellite imagery, NDVI maps, land-cover maps, and confusion matrices
Scikit-learn	Random Forest model training, prediction, and accuracy assessment
Joblib	Saving and loading trained machine learning models

5.2.3 Importing Required Libraries

The first step in the Python implementation involved importing the required libraries for geospatial data processing, numerical computation, visualization, machine learning, and model management. Each library was selected based on its specialized functionality within the land-cover classification workflow. The imported modules enabled the notebook to read and process Landsat 9 raster datasets, handle vector-based training and validation regions, perform numerical operations, visualize intermediate and final outputs, train the Random Forest classifier, evaluate classification accuracy, and save the trained machine learning model for future use.

The import statements were organized at the beginning of the notebook to ensure that all required dependencies were loaded before the execution of the workflow. This approach improves code readability, simplifies maintenance, and minimizes runtime errors caused by missing libraries.

The Python libraries imported for this implementation are shown in Code Listing 5.1.

Code Listing 5.1: Importing Required Python Libraries

```
# Numerical Computing
import numpy as np
import pandas as pd

# Visualization
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap, BoundaryNorm

# Raster Processing
import rasterio
from rasterio.plot import show
from rasterio.mask import mask

# Vector Data Processing
import geopandas as gpd

# Machine Learning
from sklearn.ensemble import RandomForestClassifier
import joblib

# Accuracy Assessment
from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay,
    classification_report,
    accuracy_score,
    cohen_kappa_score
)

# Standard Libraries
from pathlib import Path
```

The imported libraries collectively provide the foundation for the complete implementation of the land-cover classification workflow. NumPy and Pandas support numerical and tabular data manipulation, Rasterio and GeoPandas enable raster and vector data processing, Matplotlib is used for visualization, Scikit-learn provides the Random Forest classifier and evaluation metrics, and Joblib is used to save the trained machine learning model. By combining these libraries, the entire workflow can be executed programmatically within a single Python environment.

5.2.4 Project Configuration

Before processing the satellite imagery, a centralized project configuration was established to define the directory structure, input datasets, output locations, and other parameters required throughout the implementation. Configuring these settings at the beginning of the notebook ensures that all subsequent operations reference a common set of variables, making the workflow organized, reusable, and easier to maintain.

The configuration section specifies the locations of the Landsat 9 spectral bands, training and validation Region of Interest (ROI) files, and directories for storing intermediate and final outputs. By storing these values in dedicated variables, modifications can be made in a single location without changing multiple sections of the source code.

A structured directory hierarchy was also maintained to organize the generated outputs into separate folders for RGB composites, NDVI, classification results, and trained machine learning models. This organization improves project readability, simplifies file management, and ensures that the generated datasets can be easily accessed for further analysis or comparison.

The project configuration used in this implementation is presented in Code Listing 5.2.

Code Listing 5.2: Project Configuration

```
# Project Directories

PROJECT_ROOT = Path.cwd().parent.parent

QGIS_DIR = PROJECT_ROOT / "QGIS Project"
PYTHON_DIR = PROJECT_ROOT / "Python"

SUBSET_DIR = QGIS_DIR / "Subset_Data"
TRAINING_DIR = QGIS_DIR / "Classification"/ "Training_Data"
VALIDATION_DIR = QGIS_DIR / "Classification" / "Validation_Data"

OUTPUT_DIR = PYTHON_DIR / "Outputs"
RGB_OUTPUT = OUTPUT_DIR / "RGB_Composite"
NDVI_OUTPUT = OUTPUT_DIR / "NDVI"
CLASSIFICATION_OUTPUT = OUTPUT_DIR / "Classification"
MODEL_OUTPUT = OUTPUT_DIR / "Models"
```

The centralized configuration enhances the reproducibility and maintainability of the workflow by minimizing hard-coded values within the implementation. It enables the same notebook to be reused with different datasets or study areas by modifying only the configuration parameters rather than the core processing logic. This modular approach also reduces the likelihood of file path errors and supports a well-organized project structure for large-scale geospatial analysis.

5.3 Python Workflow

The Python implementation follows a systematic and modular workflow for performing land-cover classification using the Random Forest algorithm. Unlike the graphical workflow implemented in QGIS, each processing step is executed programmatically through Python scripts, enabling automation, reproducibility, and efficient handling of geospatial data. The workflow was designed to process Landsat 9 multispectral imagery from data preparation to the generation of the final classified map and accuracy assessment.

The implementation begins by loading the Landsat 9 subset bands and combining them into a multispectral band stack. This band stack serves as the primary input for all subsequent image processing operations. The workflow then generates the True Color Composite (RGB) and False Color Composite (FCC) images to facilitate visual interpretation of the study area, followed by the computation of the Normalized Difference Vegetation Index (NDVI) for vegetation analysis.

Training Region of Interest (ROI) polygons are subsequently rasterized and used to extract representative training samples from the multispectral image. These samples are converted into a feature matrix and corresponding class labels, which are used to train the Random Forest

classifier. After model training, the classifier is applied to every pixel in the study area to generate the land-cover classification raster. The trained model is also saved to disk, allowing it to be reused without retraining.

To evaluate the performance of the classification model, independent validation ROIs are rasterized and used to extract ground truth and predicted class labels. These labels are compared to compute the confusion matrix, overall accuracy, Cohen's Kappa coefficient, Producer's Accuracy, User's Accuracy, and the classification report. Finally, the workflow exports all generated outputs, including the classified raster, land-cover map, trained model, and accuracy assessment reports.

The complete Python workflow implemented in this study is illustrated in Figure 5.1.

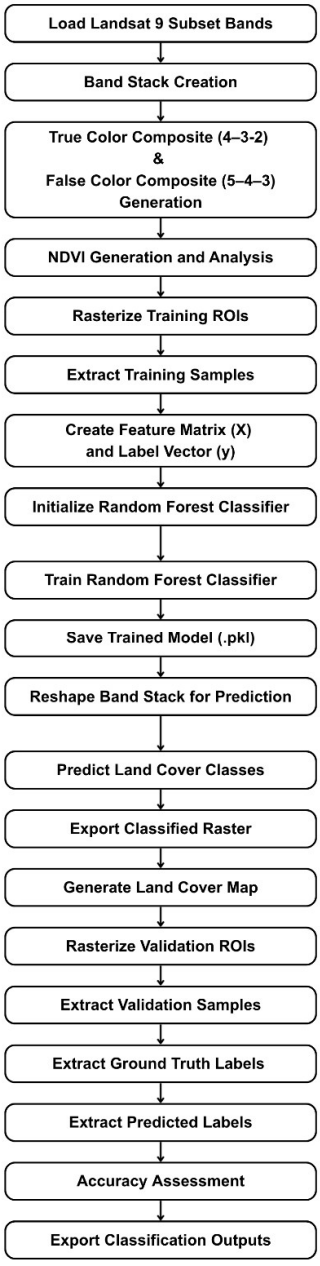


Figure 5.1: Workflow of the Python-Based Land Cover Classification Using the Random Forest Algorithm

The modular structure of the workflow ensures that each processing stage can be executed, modified, or reused independently without affecting the remaining implementation. Such an approach improves the readability, maintainability, and scalability of the workflow while providing a fully automated solution for satellite image classification. Compared with graphical GIS software, the Python workflow offers greater flexibility for parameter tuning, model optimization, batch processing, and integration with additional machine learning techniques, making it well suited for advanced remote sensing applications.

5.4 Data Loading

The Python implementation begins by loading the preprocessed Landsat 9 multispectral imagery into the working environment. To maintain consistency with the QGIS-based workflow presented in Chapter 4, the subsetted raster bands generated after clipping the original Landsat 9 scene to the Area of Interest (AOI) were used as the input dataset. Using the subsetted imagery reduces computational requirements while ensuring that all analyses are performed only within the selected study area.

The workflow utilizes six multispectral bands of the Landsat 9 Operational Land Imager (OLI), namely Band 2 (Blue), Band 3 (Green), Band 4 (Red), Band 5 (Near Infrared), Band 6 (Shortwave Infrared-1), and Band 7 (Shortwave Infrared-2). These spectral bands collectively provide the information required for image visualization, vegetation analysis, feature extraction, and supervised land-cover classification.

Each raster band was loaded using the Rasterio library, which provides efficient functions for reading georeferenced raster datasets while preserving their associated spatial metadata. After loading the spectral bands, the program verifies the availability of each dataset before proceeding to the subsequent image processing operations.

The Python implementation used for loading the Landsat 9 spectral bands is presented in Code Listing 5.3.

Code Listing 5.3: Loading Landsat 9 Spectral Bands

```
# Landsat 9 Jodhpur Subset Bands

band_2 = SUBSET_DIR / "Jodhpur_Subset_B2.tif"
band_3 = SUBSET_DIR / "Jodhpur_Subset_B3.tif"
band_4 = SUBSET_DIR / "Jodhpur_Subset_B4.tif"
band_5 = SUBSET_DIR / "Jodhpur_Subset_B5.tif"
band_6 = SUBSET_DIR / "Jodhpur_Subset_B6.tif"
band_7 = SUBSET_DIR / "Jodhpur_Subset_B7.tif"

bands = {
    "Blue (Band 2)"  : band_2,
    "Green (Band 3)" : band_3,
    "Red (Band 4)"   : band_4,
    "NIR (Band 5)"   : band_5,
    "SWIR (Band 6)"  : band_6,
    "SWIR (Band 7)"  : band_7,
}

print("-"*60)
```

```
print("LANDSAT 9 INPUT DATASET")
print("-"*60)

for name, path in bands.items():
    print(f"{name:<30} : {path.name}")

print("-"*60)

# Verify Input Files

print("Verifying input files...")

for name, path in bands.items():
    if not path.exists():
        raise FileNotFoundError(f"Input file not found: {path}")
    else:
        print(f"Input file verified: {path.name}")
```

```
------------------------------------------------------------
LANDSAT 9 INPUT DATASET
------------------------------------------------------------
Blue (Band 2)                  : Jodhpur_Subset_B2.tif
Green (Band 3)                 : Jodhpur_Subset_B3.tif
Red (Band 4)                   : Jodhpur_Subset_B4.tif
NIR (Band 5)                   : Jodhpur_Subset_B5.tif
SWIR (Band 6)                  : Jodhpur_Subset_B6.tif
SWIR (Band 7)                  : Jodhpur_Subset_B7.tif
------------------------------------------------------------
Verifying input files...
Input file verified: Jodhpur_Subset_B2.tif
Input file verified: Jodhpur_Subset_B3.tif
Input file verified: Jodhpur_Subset_B4.tif
Input file verified: Jodhpur_Subset_B5.tif
Input file verified: Jodhpur_Subset_B6.tif
Input file verified: Jodhpur_Subset_B7.tif
```

Figure 5.2: Successfully Verified Landsat 9 Spectral Bands

Successful loading of the Landsat 9 spectral bands is a prerequisite for all subsequent image processing tasks. The loaded datasets provide the multispectral information required for raster visualization, vegetation index computation, feature extraction, and Random Forest-based land-cover classification. By organizing the input datasets through a centralized configuration, the workflow becomes more reproducible and can be easily adapted to different study areas by modifying only the input file paths.

5.5 Raster Metadata

After loading the Landsat 9 spectral bands, the spatial metadata associated with the raster dataset was extracted to verify its integrity and preserve its geospatial properties throughout the processing workflow. Raster metadata provides essential information about the dataset, including its dimensions, coordinate reference system (CRS), affine transformation matrix, spatial resolution, data type, and other characteristics required for accurate geospatial analysis.

The Rasterio library was used to access the metadata stored within the raster file. This information was examined before performing any image processing operations to ensure that the input dataset had been correctly loaded and that all subsequent outputs would retain the

original geographic reference. Maintaining consistent raster metadata is particularly important when generating derived products such as RGB composites, False Color Composite (FCC) images, NDVI rasters, and classified land-cover maps, as it ensures compatibility with GIS software and other geospatial datasets.

The implementation used for reading the raster metadata is presented in Code Listing 5.4.

Code Listing 5.4: Reading Raster Metadata

```
# Read Raster Metadata

metadata_band = band_4

with rasterio.open(metadata_band) as src:
    print("-"*115)
    print("LANDSAT 9 RASTER METADATA")
    print("-"*115)

    print(f"File Name                          : {metadata_band.name}")
    print(f"Coordinate Reference System (CRS) : {src.crs}")
    print(f"Width                              : {src.width} pixels")
    print(f"Height                             : {src.height} pixels")
    print(f"Number of Bands                    : {src.count}")
    print(f"Data Type                          : {src.dtypes[0]}")
    print(f"Spatial Resolution                 : {src.res[0]} x {src.res[1]}
meters")
    print(f"Geographic Bounds                  : {src.bounds}")

    print("-"*115)
```

```
-------------------------------------------------------------------------------------------------------------------
LANDSAT 9 RASTER METADATA
-------------------------------------------------------------------------------------------------------------------
File Name                          : Jodhpur_Subset_B4.tif
Coordinate Reference System (CRS) : EPSG:32643
Width                              : 1316 pixels
Height                             : 974 pixels
Number of Bands                    : 1
Data Type                          : uint16
Spatial Resolution                 : 30.0 x 30.0 meters
Geographic Bounds                  : BoundingBox(left=282870.0, bottom=2891370.0, right=322350.0, top=2920590.0)
-------------------------------------------------------------------------------------------------------------------
```

Figure 5.3: Raster Metadata of the Landsat 9 Dataset

The extracted metadata confirms that the Landsat 9 subset has been correctly loaded into the Python environment while preserving its original spatial characteristics. This verification ensures that all subsequent processing operations are performed on a georeferenced dataset and that the generated outputs remain spatially consistent with the original imagery and the QGIS-based implementation described in the previous chapter.

5.6 Spectral Bands Preview

Before generating composite images and performing land-cover classification, the individual Landsat 9 spectral bands were visualized to verify the quality and integrity of the loaded raster data. Visual inspection of each spectral band provides an understanding of the information captured in different regions of the electromagnetic spectrum and confirms that the satellite imagery has been correctly loaded into the Python environment.

The visualization was performed using the Matplotlib library by displaying each spectral band separately with an appropriate grayscale color map. Although individual bands do not represent natural-color images, they reveal variations in surface reflectance corresponding to different land-cover features. These variations form the basis for generating multispectral composites, vegetation indices, and machine learning features used during the classification process.

The implementation used for visualizing the individual Landsat 9 spectral bands is presented in Code Listing 5.5.

Code Listing 5.5: Visualization of Individual Landsat 9 Spectral Bands

```
# Preview Landsat 9 Spectral Bands

fig, axes = plt.subplots(2, 3, figsize=(20, 10))

for ax, (title, band_path) in zip(axes.ravel(), bands.items()):
    with rasterio.open(band_path) as src:
        iamge = src.read(1)

    # ax.imshow(iamge, cmap='gray')
    vmin, vmax = np.percentile(iamge, (2, 98))
    ax.imshow(iamge, cmap='gray', vmin=vmin, vmax=vmax)
    ax.set_title(title)
    ax.axis('off')

plt.tight_layout
plt.show()
```

The six spectral bands used in this study are illustrated in Figure 5.4.

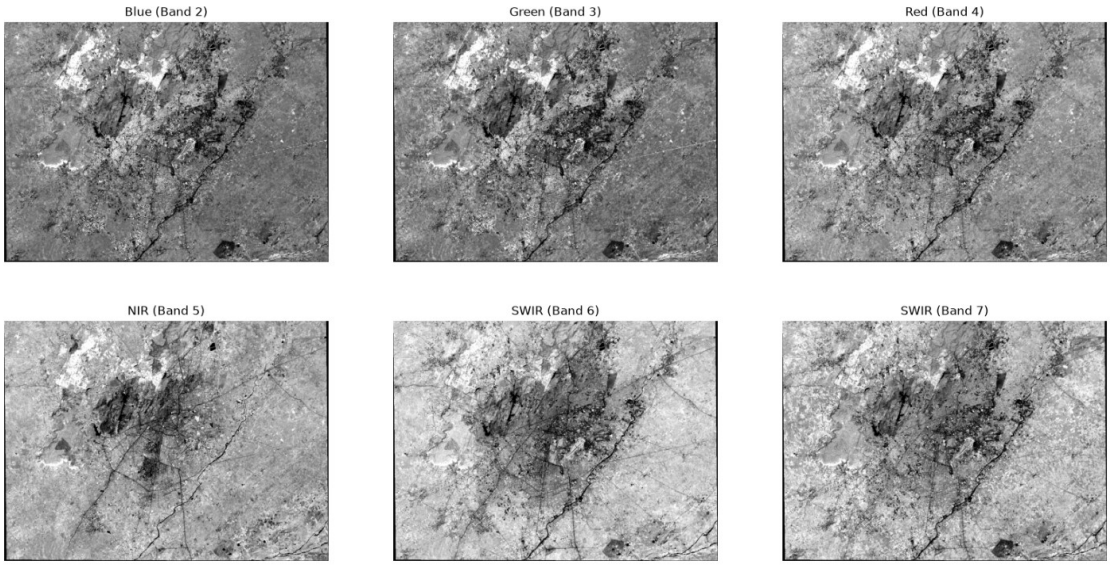


Figure 5.4: Individual Spectral Bands of the Landsat 9 Study Area

The visualization confirms that all spectral bands have been successfully loaded and contain distinct spectral information required for subsequent image processing operations. Each band highlights different surface characteristics of the study area, enabling the extraction of meaningful information during RGB and False Color Composite generation, NDVI

computation, and Random Forest-based land-cover classification. The successful visualization of the spectral bands also verifies the correctness of the input data before proceeding to the next stages of the workflow.

5.7 True Color Composite (RGB) Generation

The True Color Composite (RGB) image was generated to visualize the Landsat 9 study area using natural colors that closely resemble human visual perception. As described in Chapter 4, the RGB composite for Landsat 9 is created by assigning Band 4 (Red), Band 3 (Green), and Band 2 (Blue) to the corresponding color channels. In the Python implementation, these bands were extracted from the multispectral band stack and combined to form a three-channel RGB image.

The original raster pixel values contain a broad range of digital numbers that are not directly suitable for visualization. Therefore, each spectral band was normalized prior to display to improve image contrast and enhance the visibility of different land-cover features. After normalization, the three bands were merged into a single RGB image and displayed using the Matplotlib library. The generated composite was also exported as a high-resolution PNG image for documentation and further analysis.

The implementation used for generating the True Color Composite is presented in Code Listing 5.6.

Code Listing 5.6: Generation of the True Color Composite (RGB)

```
# Read RGB Bands

with rasterio.open(band_4) as src:
    red_band = src.read(1)

with rasterio.open(band_3) as src:
    green_band = src.read(1)

with rasterio.open(band_2) as src:
    blue_band = src.read(1)

# Apply Contrast Stretching

red_min, red_max = np.percentile(red_band, (2, 98))
green_min, green_max = np.percentile(green_band, (2, 98))
blue_min, blue_max = np.percentile(blue_band, (2, 98))

# Normalize RGB Bands

red = np.clip((red_band - red_min) / (red_max - red_min), 0, 1)
green = np.clip((green_band - green_min) / (green_max - green_min), 0, 1)
blue = np.clip((blue_band - blue_min) / (blue_max - blue_min), 0, 1)

# Create RGB Composite Image

tcc = np.dstack((red, green, blue))
```

```
# Display True Color Composite Image

plt.figure(figsize=(10, 10))
plt.imshow(tcc)

plt.title("True Color Composite (Bands 4-3-2)",
          fontsize=16,
          fontweight='bold'
)

plt.axis('off')
plt.tight_layout()

plt.show()

# Save True Color Composite Image

output_path = RGB_OUTPUT / "Jodhpur_True_Color_Composite.png"

plt.imsave(output_path, tcc)

print(f"Image saved successfully at: {output_path}")
```

The generated True Color Composite of the Jodhpur study area is shown in Figure 5.5.

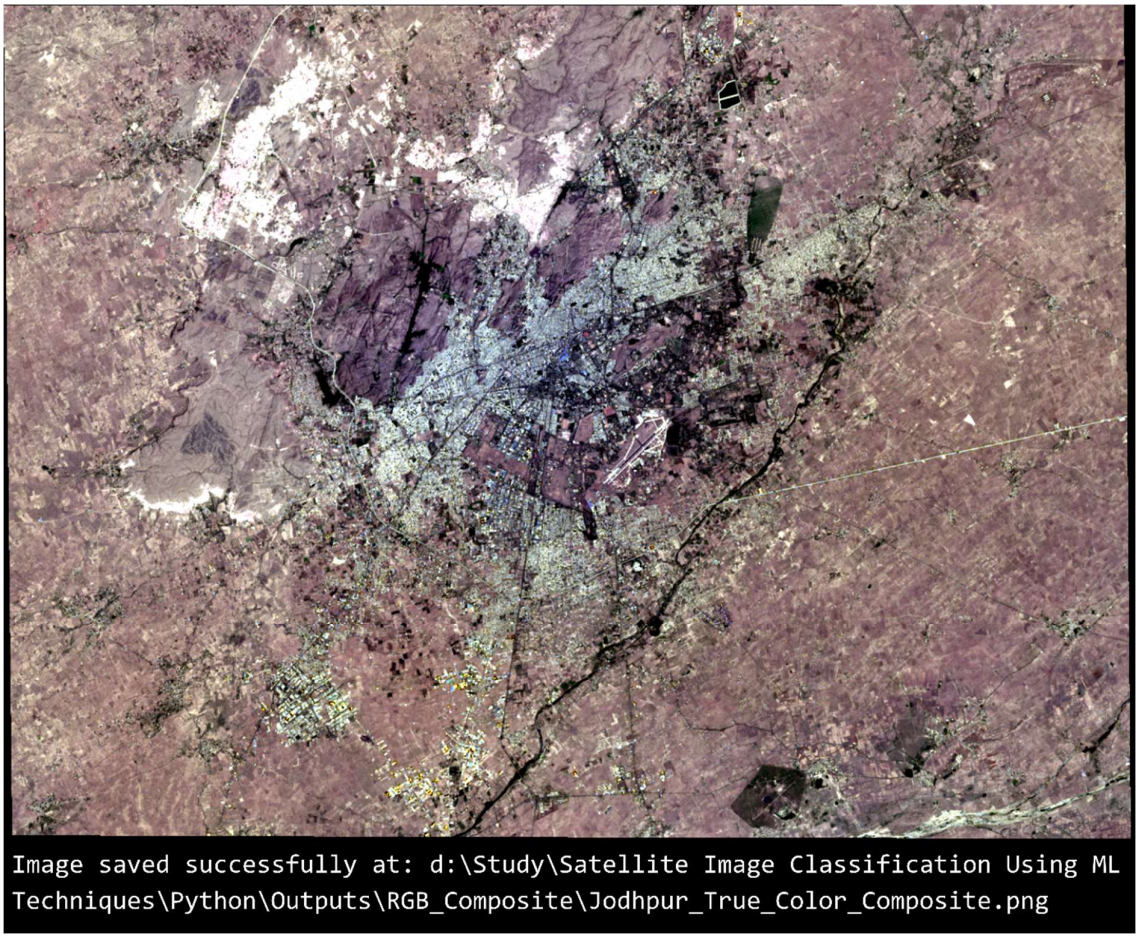


Figure 5.5: True Color Composite (RGB) Generated using Python

The generated RGB composite provides a realistic representation of the study area, making it easier to identify major land-cover features such as built-up areas, vegetation, water bodies, mining regions, and barren land through visual interpretation. Although the RGB image is primarily used for visualization, it also serves as an important reference for validating the quality of the input imagery and comparing the subsequent outputs generated during the classification workflow. The successful generation of the RGB composite confirms that the multispectral band stack has been correctly processed and is suitable for further image analysis.

5.8 False Color Composite (FCC) Generation

The False Color Composite (FCC) image was generated to enhance the visualization of vegetation and other land-cover features that are not easily distinguishable in a natural color image. As discussed in Chapter 4, the FCC for Landsat 9 is created by assigning Band 5 (Near Infrared), Band 4 (Red), and Band 3 (Green) to the red, green, and blue color channels, respectively. This band combination emphasizes healthy vegetation in shades of red due to its high reflectance in the near-infrared region of the electromagnetic spectrum.

In the Python implementation, the required spectral bands were extracted from the multispectral band stack and combined to create a three-channel False Color Composite. Similar to the RGB composite, the selected bands were normalized before visualization to improve image contrast and display quality. The normalized FCC image was then visualized using the Matplotlib library and exported as a high-resolution PNG image for documentation and further analysis.

The Python implementation used for generating the False Color Composite is presented in Code Listing 5.7.

Code Listing 5.7: Generation of the False Color Composite (FCC)

```
# Read FCC Bands

with rasterio.open(band_5) as src:
    nir_band = src.read(1)

with rasterio.open(band_4) as src:
    red_band = src.read(1)

with rasterio.open(band_3) as src:
    green_band = src.read(1)

# Apply Contrast Stretching

nir_min, nir_max = np.percentile(nir_band, (2, 98))
red_min, red_max = np.percentile(red_band, (2, 98))
green_min, green_max = np.percentile(green_band, (2, 98))

# Normalize FCC Bands

nir = np.clip((nir_band - nir_min) / (nir_max - nir_min), 0, 1)
red = np.clip((red_band - red_min) / (red_max - red_min), 0, 1)
green = np.clip((green_band - green_min) / (green_max - green_min), 0, 1)

# Create False Color Composite Image
fcc = np.dstack((nir, red, green))
```

```
# Display False Color Composite Image

plt.figure(figsize=(10, 10))
plt.imshow(fcc)

plt.title("False Color Composite (Bands 5-4-3)",
          fontsize=16,
          fontweight='bold'
)

plt.axis('off')
plt.tight_layout()

plt.show()

# Save False Color Composite Image

output_path = RGB_OUTPUT / "Jodhpur_False_Color_Composite.png"

plt.imsave(output_path, fcc)

print(f"Image saved successfully at: {output_path}")
```

The generated False Color Composite of the Jodhpur study area is shown in Figure 5.6.

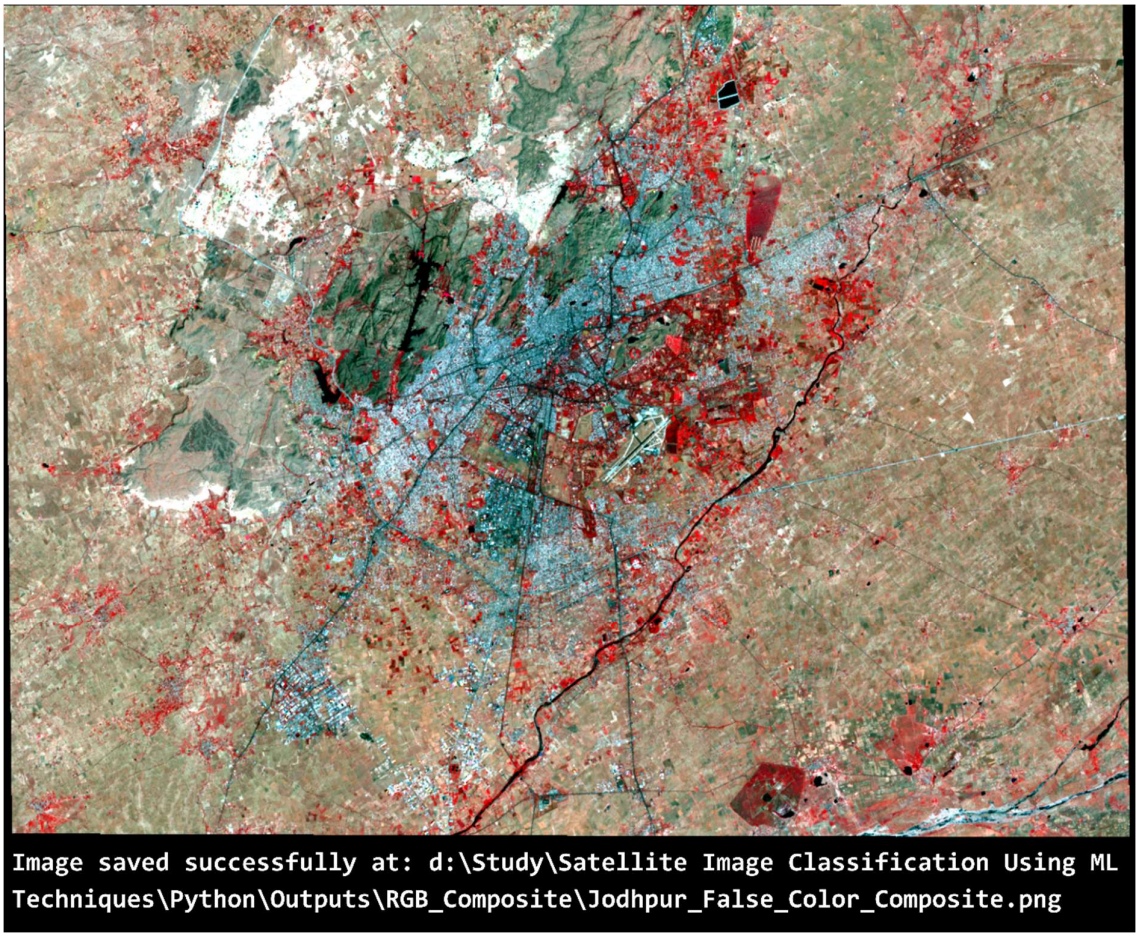


Figure 5.6: False Color Composite (FCC) Generated using Python

The generated False Color Composite clearly highlights vegetation in bright red tones, while built-up areas, mining regions, barren land, and water bodies exhibit distinct spectral responses. Compared to the True Color Composite, the FCC provides improved visual discrimination of vegetation and assists in identifying different land-cover types based on their spectral characteristics. The generated FCC image also serves as a valuable reference for interpreting the NDVI map and validating the land-cover classification results obtained in the subsequent stages of the workflow.

5.9 Normalized Difference Vegetation Index (NDVI) Generation

The Normalized Difference Vegetation Index (NDVI) was generated to assess the distribution and condition of vegetation within the study area. As discussed in Chapter 4, NDVI is one of the most widely used spectral indices in remote sensing and is calculated using the Near Infrared (NIR) and Red spectral bands. In this chapter, the emphasis is placed on the implementation of NDVI generation using Python.

The NDVI was computed using Band 5 (Near Infrared) and Band 4 (Red) of the Landsat 9 imagery. These bands were extracted from the multispectral band stack, and the NDVI value for each pixel was calculated using the standard NDVI equation. To prevent division-by-zero errors during computation, the calculation was performed only for pixels with a non-zero denominator, while pixels with a zero denominator were assigned a value of zero. The resulting NDVI values were stored as a floating-point raster while preserving the original spatial reference of the input imagery.

After computation, the NDVI raster was visualized using an appropriate color gradient to highlight variations in vegetation density across the study area. The generated raster was subsequently exported in GeoTIFF format for further geospatial analysis, and a QML style file was also added to facilitate consistent visualization within QGIS. In addition, a high-resolution PNG image of the NDVI map was exported for documentation and report preparation.

The Python implementation used for calculating and exporting the NDVI is presented in Code Listing 5.8.

Code Listing 5.8: Generation of the Normalized Difference Vegetation Index (NDVI)

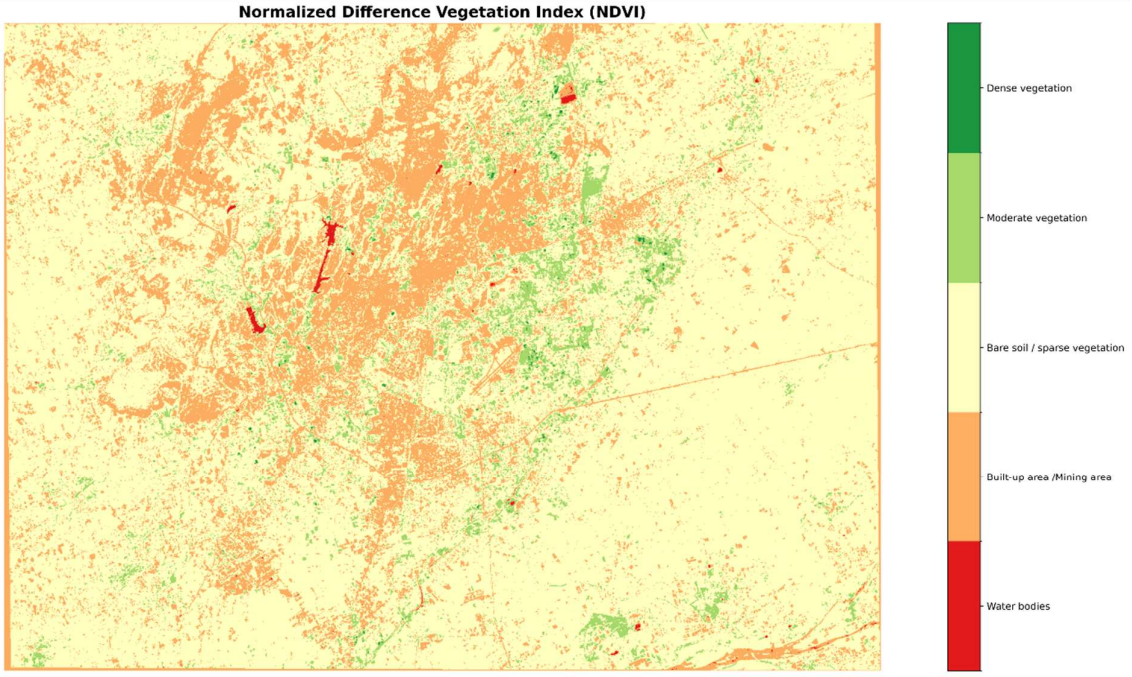
```
# Read NDVI Bands

with rasterio.open(band_5) as src:
    nir_band = src.read(1).astype(np.float32)

with rasterio.open(band_4) as src:
    red_band = src.read(1).astype(np.float32)

# Calculate NDVI

denominator = (nir_band + red_band)
```

```
ndvi = np.divide(
    nir_band - red_band,
    denominator,
    out = np.zeros_like(denominator, dtype=np.float32),
    where = denominator != 0
)

# NDVI Color Map

ndvi_colors = [
    "#E31A1C",  # Water bodies, shadows, clouds
    "#FDAE61",  # Built-up area, barren rock, mining area
    "#FFFFBF",  # Bare soil / sparse vegetation
    "#A6D96A",  # Moderate vegetation
    "#1A9641",  # Dense vegetation
]

ndvi_bounds = [-1.0, 0.0, 0.1, 0.2, 0.35, 1.0]

cmap = ListedColormap(ndvi_colors)
norm = BoundaryNorm(ndvi_bounds, cmap.N)

# Display NDVI Image

plt.figure(figsize=(20, 10))

plt.imshow(ndvi, cmap=cmap, norm=norm)

cbar = plt.colorbar(ticks=[-0.5, 0.05, 0.15, 0.275, 0.675])

cbar.ax.set_yticklabels([
    "Water bodies",
    "Built-up area /Mining area",
    "Bare soil / sparse vegetation",
    "Moderate vegetation",
    "Dense vegetation"
])

plt.title("Normalized Difference Vegetation Index (NDVI)",
          fontsize=16,
          fontweight='bold'
)

plt.axis('off')
plt.tight_layout()

# Save NDVI Image

output_path = NDVI_OUTPUT / "Jodhpur_NDVI.png"

plt.savefig(
    output_path,
    dpi=300,
    bbox_inches='tight'
)

print(f"Image saved successfully at: {output_path}")
```

```
# Show NDVI Image
plt.show()

# Save NDVI Raster

with rasterio.open(band_4) as src:
    metadata = src.meta.copy()

metadata.update(
    dtype=rasterio.float32,
    count=1,
)

output_path = NDVI_OUTPUT / "Jodhpur_NDVI.tif"

with rasterio.open(
    output_path,
    'w',
    **metadata
) as dst:
    dst.write(ndvi.astype(rasterio.float32), 1)

print(f"Raster saved successfully at: {output_path}")
```

The generated NDVI map of the Jodhpur study area is shown in Figure 5.7.

```
Image saved successfully at: d:\Study\Satellite Image Classification Using ML
Techniques\Python\Outputs\RGB_Composite\Jodhpur_False_Color_Composite.png
```




```
Raster saved successfully at: d:\Study\Satellite Image Classification Using ML
Techniques\Python\Outputs\NDVI\Jodhpur_NDVI.tif
```

Figure 5.7: Normalized Difference Vegetation Index (NDVI) Generated using Python

The generated NDVI map clearly distinguishes vegetated and non-vegetated regions based on their spectral response. Areas with higher NDVI values correspond to healthy vegetation, whereas lower or negative values represent built-up areas, barren land, mining regions, and water bodies. The exported NDVI raster and visualization provide an additional layer of information for interpreting land-cover characteristics and serve as a useful reference during the subsequent Random Forest classification process.

5.10 Training and Validation Data Preparation

The preparation of an accurate and representative training dataset is an essential step in supervised land-cover classification. Before training the Random Forest classifier, the training and validation Region of Interest (ROI) datasets created during the QGIS workflow were imported into the Python environment. These vector datasets contain manually labeled samples representing the five land-cover classes considered in this study: Built-up Area, Vegetation, Water Bodies, Mining Area, and Other Area.

The training ROIs were used to extract representative spectral information for model training, while the validation ROIs were reserved exclusively for evaluating the performance of the trained classifier. Separating the training and validation datasets ensures an unbiased assessment of the classification results and improves the reliability of the accuracy evaluation.

The complete workflow for preparing the training dataset consists of stacking the spectral bands, verifying the coordinate reference system, rasterizing the training polygons, extracting spectral samples, and organizing the extracted values into a feature matrix suitable for machine learning. These steps are described in the following sections.

5.10.1 Loading Training and Validation ROIs

The training and validation datasets were stored as vector layers generated during the QGIS-based implementation. These datasets were imported into the Python environment using the GeoPandas library, which provides efficient functions for reading and manipulating geospatial vector data.

After loading the datasets, their attributes and geometry were verified to ensure that all land-cover classes were correctly represented before further processing. The imported ROIs serve as the reference data for supervised learning and subsequent accuracy assessment.

The implementation used for loading the training and validation ROI datasets is presented in Code Listing 5.9.

Code Listing 5.9: Loading Training and Validation ROI Datasets

```
# Load Training and Validation ROIs

training_roi = gpd.read_file(TRAINING_DIR / "Training_ROI.gpkg")
validation_roi = gpd.read_file(VALIDATION_DIR / "Validation_ROI.gpkg")

print("Training and Validation ROIs loaded successfully.")

# ROI Information
print(f"Training Freatures   : {len(training_roi)}")
print(f"Validation Freatures : {len(validation_roi)}")
print()
```

```
print(f"Training CRS   : {training_roi.crs}")
print(f"Validation CRS : {validation_roi.crs}")

# Preview Training and Validation ROIs

fig, axes = plt.subplots(1, 2, figsize=(20, 10))

training_roi.plot(
    ax=axes[0],
    column="class_info",
    legend=True
)

axes[0].set_title("Training ROIs")
axes[0].set_axis_off()

validation_roi.plot(
    ax=axes[1],
    column="class_info",
    legend=True
)

axes[1].set_title("Validation ROIs")
axes[1].set_axis_off()

plt.tight_layout()

plt.show()
```

```
Training and Validation ROIs loaded successfully.

Training Freatures  : 151
Validation Freatures : 47

Training CRS  : EPSG:32643
Validation CRS : EPSG:32643
```

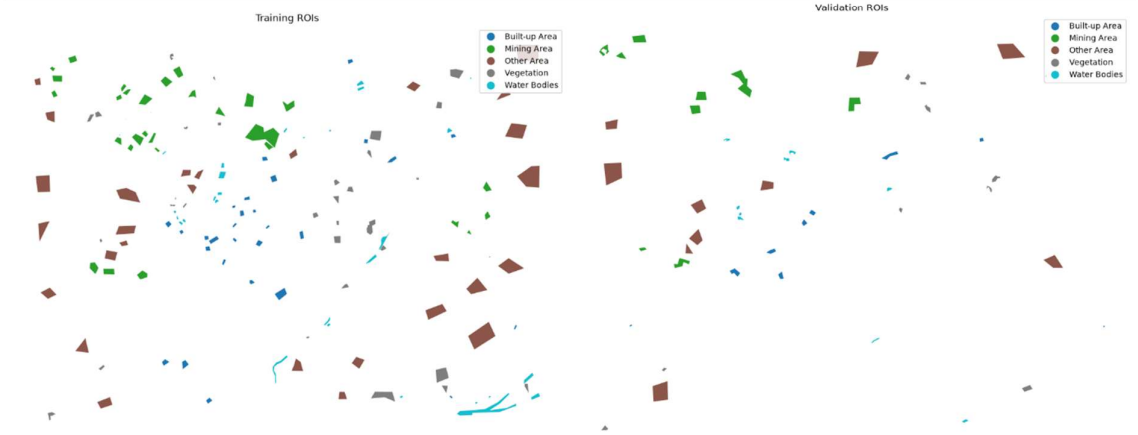


Figure 5.8: Training and Validation ROI Datasets Loaded in Python

The successful loading of the ROI datasets confirms that the labeled reference data are available for preparing the training samples required by the Random Forest classifier.

5.10.2 Preparation of Training Samples

The imported training ROIs were subsequently processed to create the dataset required for supervised machine learning. Initially, the individual Landsat 9 spectral bands were combined to form a multispectral band stack. This structure enables simultaneous access to the spectral information of all bands during feature extraction.

Before rasterization, the coordinate reference systems of both the raster imagery and the vector ROIs were verified to ensure spatial consistency. The training polygons were then rasterized to produce a raster mask having the same dimensions, resolution, and spatial extent as the Landsat image. Each pixel within the rasterized polygons was assigned its corresponding land-cover class identifier.

Using the generated raster mask, spectral values were extracted from all Landsat bands for every labeled pixel. The extracted values were organized into a feature matrix (X), while the associated class identifiers were stored in a target vector (y). These datasets constitute the training samples used for Random Forest model development.

The implementation used for stacking the spectral bands, verifying the coordinate reference system, rasterizing the training ROIs, and extracting the training samples is presented in Code Listing 5.10.

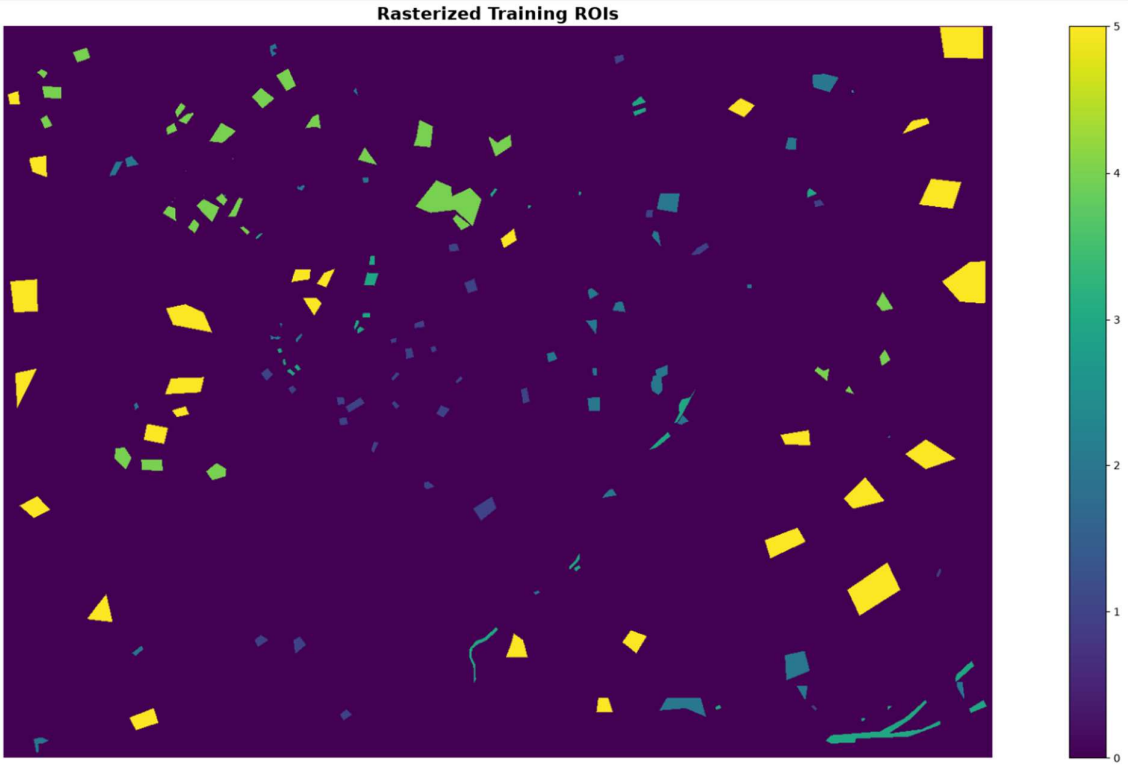
Code Listing 5.10: Preparation of Training Samples

```
# Stack Spectral Bands

band_stack = []

for band_path in bands.values():
    with rasterio.open(band_path) as src:
        band_stack.append(src.read(1))

band_stack = np.stack(band_stack)

print(f"Band Stack Shape: {band_stack.shape}")

# Verify CRS

with rasterio.open(band_4) as src:
    raster_crs = src.crs

print(f"Raster CRS: {raster_crs}")
print(f"Training ROI CRS: {training_roi.crs}")
print(f"Validation ROI CRS: {validation_roi.crs}")

# Check CRS Consistency

if(
    raster_crs == training_roi.crs
    and raster_crs == validation_roi.crs
):
    print("CRS consistency verified.")
else:
    print("CRS inconsistency detected.")
```

```
# Copy the raster's transform and shape for later use

with rasterio.open(band_4) as src:
    transform = src.transform
    raster_shape = (src.height, src.width)

# Create Geometry-Class Pairs for Training ROIs

shapes = [
    (geometry, class_id)

    for geometry, class_id in zip(
        training_roi.geometry,
        training_roi.class_id
    )
]

# Raterize Training ROIs

training_mask = rasterio.features.rasterize(
    shapes=shapes,
    out_shape=raster_shape,
    transform=transform,
    fill=0,  # Background value
    dtype=np.uint8
)

# Display Training Mask

plt.figure(figsize=(20, 10))

plt.imshow(training_mask)

plt.title("Rasterized Training ROIs",
          fontsize=16,
          fontweight='bold'
)

plt.colorbar()
plt.axis('off')
plt.tight_layout()
plt.show()

# Extract Training Samples
training_samples = training_mask > 0

# Create Feature Matrix (X) and Label Vector (y) for Training Samples
X = band_stack[:, training_samples].T
y = training_mask[training_samples]

# Display Dataset Information

print(f"Feature Matrix Shape (X) : {X.shape}")
print(f"Label Vector Shape (y)   : {y.shape}")
print(f"Number of Classes        : {len(np.unique(y))}")
print(f"Number of Samples        : {len(y)}")
```

```
# Display Class Distribution

unique_classes, class_counts = np.unique(
    y,
    return_counts=True
)

for class_id, count in zip(unique_classes, class_counts):
    print(f"Class ID {class_id:<3} : {count} samples")
```

```
Band Stack Shape: (6, 974, 1316)

Raster CRS: EPSG:32643
Training ROI CRS: EPSG:32643
Validation ROI CRS: EPSG:32643

CRS consistency verified.
```



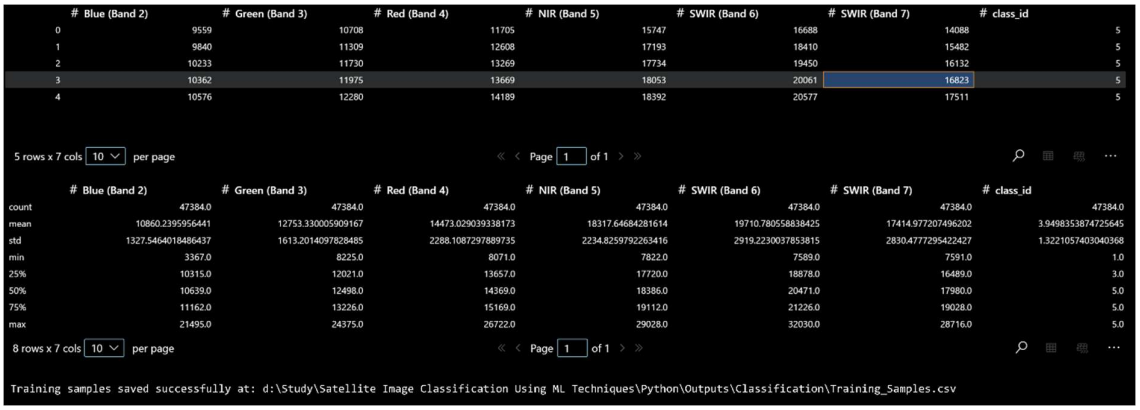

```
Feature Matrix Shape (X) : (47384, 6)
Label Vector Shape (y)  : (47384,)
Number of Classes       : 5
Number of Samples       : 47384

Class ID 1  : 3377 samples
Class ID 2  : 6175 samples
Class ID 3  : 3873 samples
Class ID 4  : 9982 samples
Class ID 5  : 23977 samples
```

Figure 5.9: Preparation of Training Samples

After extracting the samples, a preview of the generated feature matrix was examined to verify that the spectral values and corresponding class labels had been correctly prepared for machine learning.

Code Listing 5.11: Preview of Extracted Training Samples

```
# Create Training Samples DataFrame

training_samples_df = pd.DataFrame(
    X,
    columns=[
        "Blue (Band 2)",
        "Green (Band 3)",
        "Red (Band 4)",
        "NIR (Band 5)",
        "SWIR (Band 6)",
        "SWIR (Band 7)"
    ]
)

training_samples_df["class_id"] = y

# Preview Extracted Training Samples DataFrame

training_samples_df.head()

# Dataset Summary Statistics

training_samples_df.describe()

# Save Training Samples DataFrame to CSV

output_path = CLASSIFICATION_OUTPUT / "Training_Samples.csv"

training_samples_df.to_csv(output_path, index=False)

print(f"Training samples saved successfully at: {output_path}")
```



	# Blue (Band 2)	# Green (Band 3)	# Red (Band 4)	# NIR (Band 5)	# SWIR (Band 6)	# SWIR (Band 7)	# class_id
0	9559	10708	11705	15747	16688	14088	5
1	9840	11309	12608	17193	18410	15482	5
2	10233	11730	13269	17734	19450	16132	5
3	10362	11975	13669	18053	20061	16823	5
4	10576	12280	14189	18392	20577	17511	5

5 rows x 7 cols 10 per page Page 1 of 1

	# Blue (Band 2)	# Green (Band 3)	# Red (Band 4)	# NIR (Band 5)	# SWIR (Band 6)	# SWIR (Band 7)	# class_id
count	47384.0	47384.0	47384.0	47384.0	47384.0	47384.0	47384.0
mean	10860.2395956441	12753.330005909167	14473.029039338173	18317.64684281614	19710.780558838425	17414.977207496202	3.9498353874725645
std	1327.5464018486437	1613.2014097828485	2288.1087297889735	2234.8259792263416	2919.2230037853815	2830.4777295422427	1.3221057403040368
min	3367.0	8225.0	8071.0	7822.0	7589.0	7591.0	1.0
25%	10315.0	12021.0	13657.0	17720.0	18878.0	16489.0	3.0
50%	10639.0	12498.0	14369.0	18386.0	20471.0	17980.0	5.0
75%	11162.0	13226.0	15169.0	19112.0	21226.0	19028.0	5.0
max	21495.0	24375.0	26722.0	29028.0	32030.0	28716.0	5.0

8 rows x 7 cols 10 per page Page 1 of 1

```
Training samples saved successfully at: d:\Study\Satellite Image Classification Using ML Techniques\Python\Outputs\Classification\Training_Samples.csv
```

Figure 5.10: Preview of the Extracted Training Samples

The extracted training samples form the input dataset for the Random Forest classifier. Each sample contains the spectral reflectance values of the Landsat 9 bands together with its corresponding land-cover label, enabling the classifier to learn the spectral characteristics of the different land-cover classes before performing classification of the entire study area.

5.11 Random Forest Model Development

Random Forest was selected as the classification algorithm for the Python implementation based on the comparative analysis presented in Chapter 4. Among the evaluated supervised classification techniques, Random Forest achieved the highest classification accuracy and demonstrated excellent capability in distinguishing the five land-cover classes of the study area. Its ensemble learning approach, which combines the predictions of multiple decision trees, makes it well suited for multispectral satellite image classification by reducing overfitting and improving classification reliability.

To determine an appropriate model configuration, several values of the **n_estimators** parameter were evaluated. The number of decision trees directly influences the classification performance and computational cost of the Random Forest algorithm. Models containing 100, 200, 300, 400, 500, 1000, 2000, and 5000 decision trees were trained and compared using Out-of-Bag (OOB) Score, Validation Accuracy, Cohen's Kappa Coefficient, and training time.

The performance comparison of the evaluated models is presented in Table 5.3.

Table 5.3: Performance Comparison of Random Forest Models with Different Numbers of Decision Trees

Number of Decision Trees	Training Time	OOB Score	Validation Accuracy (%)	Kappa Coefficient
100	3.1 s	0.9241	92.71	0.8558
200	6.5 s	0.9254	91.81	0.8578
300	**8.7 s**	**0.9260**	**91.92**	**0.8595**
400	11.7 s	0.9263	91.84	0.8582
500	16.2 s	0.9263	91.92	0.8595
1000	34.5 s	0.9266	91.91	0.8594
2000	1 min 16.7 s	0.9269	91.94	0.8599
5000	5 min 26.7 s	0.9270	91.94	0.8599

The experimental results indicate that increasing the number of decision trees generally improved the stability of the classifier. However, beyond **300 trees**, the improvement in validation accuracy and Kappa coefficient became marginal, while the training time increased considerably. Although models containing 1000, 2000, and 5000 trees achieved slightly higher OOB scores, their classification accuracy remained nearly unchanged compared with the 300-tree model. Therefore, the Random Forest classifier configured with **300 decision trees** was selected for the remainder of the implementation, as it provided an effective balance between computational efficiency and classification performance.

The final Random Forest classifier was implemented using the **RandomForestClassifier** class provided by the Scikit-learn library. The selected hyperparameters, including the number of decision trees and the random seed, were specified before training the model using the extracted spectral samples.

The implementation used for configuring the final Random Forest classifier is presented in Code Listing 5.12.

Code Listing 5.12: Configuration of the Final Random Forest Classifier

```
# Create Random Forest Classifier

rf_classifier = RandomForestClassifier(
    n_estimators=300,
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_features='sqrt',
    bootstrap=True,
    oob_score=True,
    random_state=42,
    n_jobs=-1
)
```

After configuring the classifier, the model was trained using the complete training dataset consisting of the extracted spectral features and their corresponding land-cover labels. During training, the Random Forest algorithm automatically generated multiple decision trees from randomly selected subsets of the training samples and spectral features. The final classification model was obtained by combining the predictions of all decision trees through majority voting.

The implementation used for training the Random Forest classifier is presented in Code Listing 5.13.

Code Listing 5.13: Training the Random Forest Classifier

```
# Train Random Forest Classifier

rf_classifier.fit(X, y)
```

Following successful model training, the trained classifier was saved using the Joblib library. Saving the trained model eliminates the need for repeated training and enables the classifier to be reused for future land-cover prediction or further analysis.

The implementation used for saving the trained Random Forest model is presented in Code Listing 5.14.

Code Listing 5.14: Saving the Trained Random Forest Model

```
# Save Trained Model

model_path = MODEL_OUTPUT / "Random_Forest_Model.pkl"

joblib.dump(rf_classifier, model_path)
```

The trained Random Forest model serves as the core component of the Python-based land-cover classification workflow. The saved model is subsequently used to classify every pixel in the Landsat 9 study area, generating the final land-cover classification raster presented in the next section.

5.12 Land-Cover Classification

After successfully training the Random Forest classifier, the model was applied to the complete Landsat 9 multispectral image to generate the land-cover classification map of the study area. The objective of this step was to assign each image pixel to one of the five predefined land-cover classes based on its spectral characteristics learned during the training phase.

Before prediction, the multispectral band stack was reshaped into a two-dimensional feature matrix in which each row represented an individual pixel and each column corresponded to the reflectance value of a spectral band. This transformation converts the raster image into a format compatible with the Scikit-learn machine learning framework while preserving the spectral information of every pixel.

The trained Random Forest model was then used to predict the land-cover class of each pixel. After prediction, the one-dimensional array of predicted labels was reshaped back to the original raster dimensions, thereby generating the classified land-cover image. The classified raster preserved the spatial resolution, coordinate reference system, affine transformation, and other geospatial metadata of the original Landsat 9 imagery, ensuring compatibility with GIS software and subsequent analysis.

The implementation used for predicting the land-cover classes and generating the classified raster is presented in Code Listing 5.15.

Code Listing 5.15: Prediction of Land-Cover Classes Using the Trained Random Forest Model

```
# Reshape Band Stack for Prediction

rows, cols = band_stack.shape[1:]

prediction_data = band_stack.reshape(
    band_stack.shape[0],
    -1
).T

print(f"Prediction Data Shape: {prediction_data.shape}")

# Predict Land Cover Classes for Entire Image

predicted_classes = rf_classifier.predict(prediction_data)

print("Land cover classification completed successfully.")

# Reshape Predicted Classes to Original Image Dimensions

classified_image = predicted_classes.reshape(rows, cols)

print(f"Classified Image Shape: {classified_image.shape}")
```

```
Prediction Data Shape: (1281784, 6)
Land cover classification completed successfully.
Classified Image Shape: (974, 1316)
```

Figure 5.11: Prediction of Land-Cover Classes Using the Trained Random Forest Model

After generating the classified raster, it was exported in **GeoTIFF (.tif)** format using the metadata of the original Landsat 9 dataset. A corresponding **QML style file** was also added to preserve the land-cover symbology when the raster is opened in QGIS. In addition, a high-resolution PNG image of the classified map was exported for visualization and documentation.

The implementation used for exporting the classification results is presented in Code Listing 5.16.

Code Listing 5.16: Exporting the Classified Raster and Visualization

```
# Copy Raster Metadata for Saving Classified Image

with rasterio.open(band_4) as src:
    metadata = src.meta.copy()

# Update Raster Metadata for Classified Image

metadata.update(
    driver='GTiff',
    dtype=rasterio.uint8,
    count=1,
)

# Save Classified Raster

output_path = CLASSIFICATION_OUTPUT / "Jodhpur_Land_Cover_Classification.tif"

with rasterio.open(
    output_path,
    'w',
    **metadata
) as dst:
    dst.write(classified_image.astype(rasterio.uint8), 1)

print(f"Classified raster saved successfully at: {output_path}")

# Define Land Cover Color Map

landcover_colors = [
    "#E31A1C", # Built-up Area
    "#33A02C", # Vegetation
    "#1F78B4", # Water Bodies
    "#5A0080", # Mining Area
    "#F6FF00"  # Other Area
]

landcover_labels = [
    "Built-up Area",
    "Vegetation",
    "Water Bodies",
    "Mining Area",
    "Other Area"
]

cmap = ListedColormap(landcover_colors)
bounds = [1, 2, 3, 4, 5, 6]

norm = BoundaryNorm(bounds, cmap.N)
```

```
# Display Classified Image with Color Map

plt.figure(figsize=(20, 10))

image = plt.imshow(
    classified_image,
    cmap=cmap,
    norm=norm
)

cbar = plt.colorbar(
    image,
    ticks=[1.5, 2.5, 3.5, 4.5, 5.5],
    fraction=0.046,
    pad=0.04
)

cbar.ax.set_yticklabels(landcover_labels)

plt.title(
    "Land Cover Classification Using Random Forest",
    fontsize=16,
    fontweight='bold'
)

plt.axis('off')
plt.tight_layout()

# Save Classified Image with Color Map

output_path = CLASSIFICATION_OUTPUT / "Jodhpur_Land_Cover_Classification.png"

plt.savefig(
    output_path,
    dpi=300,
    bbox_inches='tight'
)

print(f"Classified image saved successfully at: {output_path}")

# Show Classified Image with Color Map
plt.show()
```

The final land-cover classification map generated using the Random Forest classifier is shown in Figure 5.12.

```
Classified raster saved successfully at: d:\Study\Satellite Image Classification Using ML Techniques\Python\Outputs\Classification\Jodhpur_Land_Cover_Classification.tif
Classified image saved successfully at: d:\Study\Satellite Image Classification Using ML Techniques\Python\Outputs\Classification\Jodhpur_Land_Cover_Classification.png
```

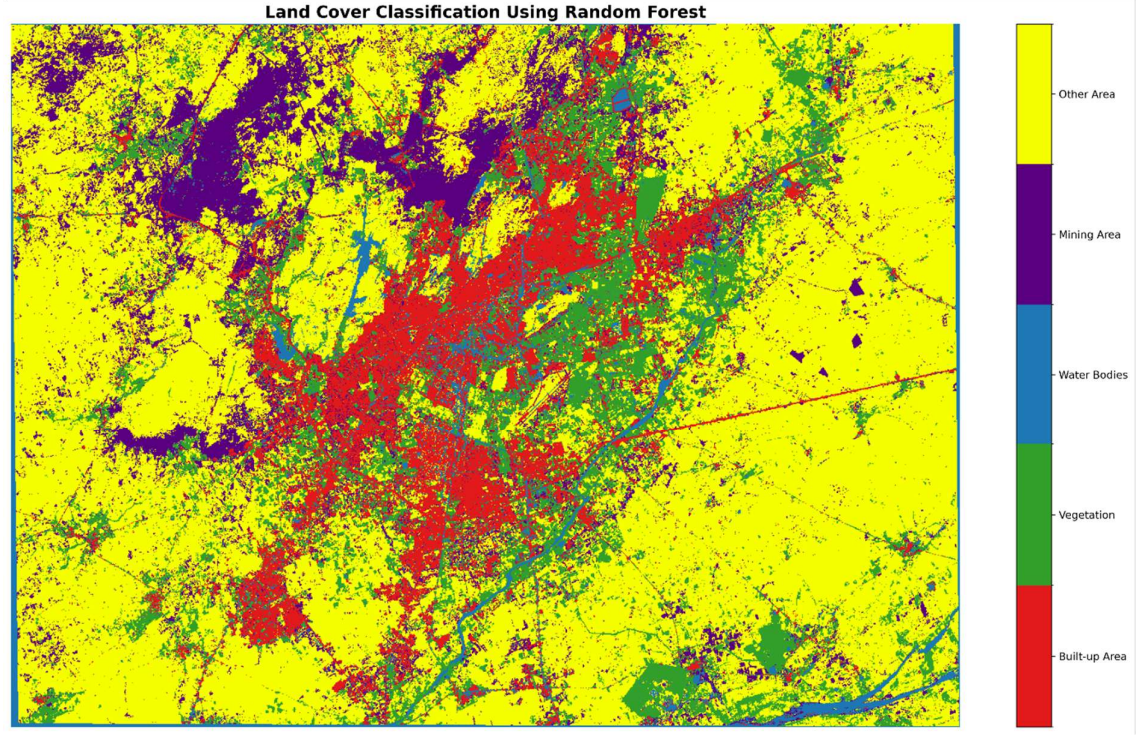


Figure 5.12: Land-Cover Classification Map Generated Using the Random Forest Classifier

The generated land-cover map successfully classifies the study area into five thematic classes: Built-up Area, Vegetation, Water Bodies, Mining Area, and Other Area. Each class is represented using a distinct color scheme to facilitate visual interpretation of the spatial distribution of land-cover features. The exported GeoTIFF and QML files enable the classified map to be readily integrated into GIS software for further visualization and spatial analysis.

5.13 Accuracy Assessment

The performance of the Random Forest classifier was evaluated using an independent validation dataset prepared during the QGIS workflow. Unlike the training samples used to build the classification model, the validation samples were not used during model training, thereby providing an unbiased assessment of the classifier's predictive performance.

Initially, the validation ROI polygons were rasterized to match the spatial resolution, dimensions, and coordinate reference system of the classified raster. The rasterized validation mask was then used to extract the corresponding predicted land-cover classes from the classified image. These predicted labels were compared with the reference class labels to quantify the classification performance.

The evaluation was carried out using several standard accuracy assessment metrics, including the Confusion Matrix, Overall Accuracy, Cohen's Kappa Coefficient, Producer's Accuracy,

User's Accuracy, and the Classification Report. Together, these metrics provide a comprehensive assessment of the classifier by measuring both overall performance and class-wise prediction accuracy.

The implementation used for preparing the validation data and performing the accuracy assessment is presented in Code Listing 5.17.

Code Listing 5.17: Accuracy Assessment of the Random Forest Classification

```
# Rasterize Validation ROIs

shapes = [
    (geometry, class_id)
    for geometry, class_id in zip(
        validation_roi.geometry,
        validation_roi.class_id
    )
]

validation_mask = rasterio.features.rasterize(
    shapes=shapes,
    out_shape=raster_shape,
    transform=transform,
    fill=0, # Background Value
    dtype=np.uint8
)

# Display Validation Mask

plt.figure(figsize=(20,10))

plt.imshow(validation_mask)

plt.title(
    "Rasterized Validation ROIs",
    fontsize=16,
    fontweight='bold'
)

plt.colorbar()
plt.axis('off')
plt.tight_layout()
plt.show()

# Extract Validation Samples

validation_samples = validation_mask > 0

# Extract Ground Truth Labels

y_true = validation_mask[validation_samples]

# Extract Predicted Labels
y_pred = classified_image[validation_samples]

# Generate Confusion Matrix

conf_matrix = confusion_matrix(y_true, y_pred)
```

```
# Display Confusion Matrix

class_labels = landcover_labels

disp = ConfusionMatrixDisplay(
    confusion_matrix=conf_matrix,
    display_labels=class_labels
)

disp.plot(
    cmap="Blues",
    values_format='d'
)

plt.title(
    "Confusion Matrix for Land Cover Classification",
    fontsize=16,
    fontweight='bold'
)

plt.xticks(rotation=90, ha='right')
plt.tight_layout()

# Save Confusion Matrix Image

output_path = CLASSIFICATION_OUTPUT / "Jodhpur_Confusion_Matrix.png"

plt.savefig(
    output_path,
    dpi=300,
    bbox_inches='tight'
)

plt.show()

# Save Confusion Matrix Values

conf_matrix_df = pd.DataFrame(
    conf_matrix,
    index=class_labels,
    columns=class_labels
)

output_path = CLASSIFICATION_OUTPUT / "Jodhpur_Confusion_Matrix.csv"

conf_matrix_df.to_csv(output_path, index=True)

# Calculate Overall Accuracy
overall_accuracy = accuracy_score(y_true, y_pred)

# Calculate Cohen's Kappa Score
kappa = cohen_kappa_score(y_true, y_pred)

# Caclulate Producer's Accuracy

producer_accuracy = (
    np.diag(conf_matrix) / np.sum(conf_matrix, axis=1)
)
```

```
# Caculate User's Accuracy

user_accuracy = (
    np.diag(conf_matrix) / np.sum(conf_matrix, axis=0)
)

# Save Accuracy Assessment Results

accuracy_assessment_df = pd.DataFrame({
    "Class" : class_labels,
    "Producer's Accuracy (%)" : producer_accuracy * 100,
    "User's Accuracy (%)" : user_accuracy * 100
})

summary_df = pd.DataFrame({
    "Metric": [
        "Overall Accuracy (%)",
        "Kappa Coefficient"
    ],
    "Value": [
        overall_accuracy * 100,
        kappa
    ]
})

output_path = CLASSIFICATION_OUTPUT / "Jodhpur_Accuracy_Assessment.csv"

with open(output_path, 'w', newline='') as file:
    summary_df.to_csv(file, index=False)
    file.write("\n")
    accuracy_assessment_df.to_csv(file, index=False)

# Generate Classification Report

report = classification_report(
    y_true,
    y_pred,
    target_names=class_labels,
    output_dict=True
)

classification_report_df = pd.DataFrame(report).transpose()

classification_report_df

# Save Classification Report

output_path = CLASSIFICATION_OUTPUT / "Jodhpur_Classification_Report.csv"

classification_report_df.to_csv(output_path, index=True)
```

The confusion matrix obtained from the validation dataset is presented in Figure 5.13.

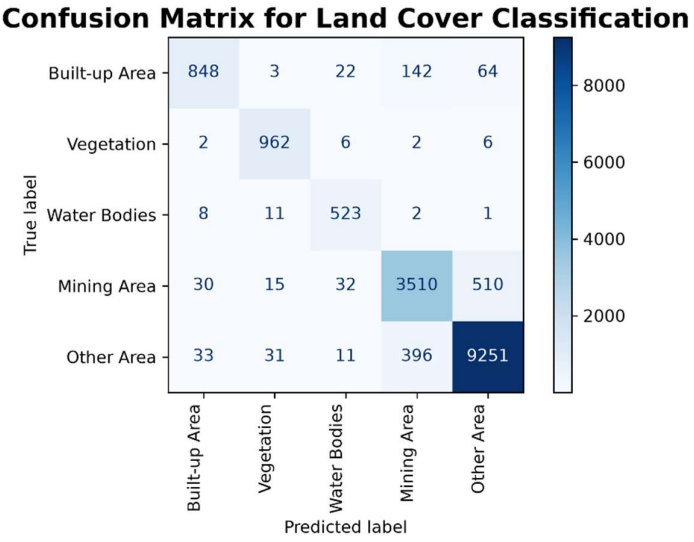


Figure 5.13: Confusion Matrix of the Random Forest Classification

The overall performance of the Random Forest classifier is summarized in Table 5.4, which presents the Overall Accuracy and Cohen's Kappa Coefficient computed using the independent validation dataset.

Table 5.4: Accuracy Assessment Results of the Random Forest Classifier

Metric	Value
Overall Accuracy	**91.92%**
Cohen's Kappa Coefficient	**0.8595**

The class-wise classification performance of the Random Forest classifier is presented in Table 5.5, which includes the precision, recall, F1-score, and support for each land-cover class.

Table 5.5: Classification Report of the Random Forest Classifier

Land-Cover Class	Precision	Recall	F1-Score	Support
Built-up Area	0.921	0.786	0.848	1,079
Vegetation	0.941	0.984	0.962	978
Water Bodies	0.880	0.960	0.918	545
Mining Area	0.866	0.857	0.861	4,097
Other Area / Barren Land	0.941	0.952	0.946	9,722
Overall Accuracy	—	—	**0.9192**	**16,421**
Macro Average	**0.910**	**0.907**	**0.907**	**16,421**
Weighted Average	**0.919**	**0.919**	**0.919**	**16,421**

The classification report indicates that the Random Forest classifier achieved high precision, recall, and F1-score values for most land-cover classes. Vegetation exhibited the highest classification performance, while the Built-up Area class showed comparatively lower recall due to spectral similarity with surrounding land-cover types. Overall, the high weighted average precision, recall, and F1-score values demonstrate the effectiveness of the developed classifier in accurately mapping the land-cover classes within the study area.

The class-wise Producer's Accuracy and User's Accuracy derived from the confusion matrix are presented in Table 5.6.

Table 5.6: Producer's Accuracy and User's Accuracy for Individual Land-Cover Classes

Land-Cover Class	Producer's Accuracy (%)	User's Accuracy (%)
Built-up Area	78.59	92.07
Vegetation	98.36	94.13
Water Bodies	95.96	88.05
Mining Area	85.67	86.62
Other Area / Barren Land	95.16	94.09

The Producer's Accuracy and User's Accuracy provide a class-wise evaluation of the classification performance. Vegetation achieved the highest Producer's Accuracy (98.36%), indicating that most vegetation pixels were correctly classified, whereas Built-up Area exhibited the lowest Producer's Accuracy (78.59%), suggesting comparatively greater confusion with other land-cover classes. Nevertheless, the high Producer's and User's Accuracy values obtained for all classes indicate that the Random Forest classifier effectively distinguished the major land-cover types within the study area. Overall, the classifier achieved an **Overall Accuracy of 91.92%** with a **Cohen's Kappa Coefficient of 0.8595**, demonstrating a high level of agreement between the predicted and reference land-cover classes. Furthermore, the classification report and the class-wise accuracy measures confirm the reliability and effectiveness of the developed Python-based land-cover classification workflow.

5.14 Comparison Between QGIS and Python Implementation

The complete land-cover classification workflow was implemented using both QGIS and Python to evaluate their effectiveness for processing Landsat 9 satellite imagery. Although both approaches followed the same fundamental workflow, including image preprocessing, visualization, vegetation analysis, supervised classification, and accuracy assessment, they differed in terms of implementation methodology, flexibility, automation, and computational capabilities.

The QGIS implementation relied primarily on the Semi-Automatic Classification Plugin (SCP), which provides an interactive graphical interface for performing remote sensing operations. In contrast, the Python implementation employed an automated workflow

developed using open-source geospatial and machine learning libraries, enabling greater control over data processing, model development, and output generation.

To ensure a fair comparison, the same Area of Interest (AOI), Landsat 9 imagery, land-cover classes, and training and validation datasets were used in both implementations. The comparative analysis of the two approaches is presented in Table 5.7.

Table 5.7: Comparison Between QGIS and Python Implementation

Parameter	QGIS Implementation	Python Implementation
Development Environment	QGIS with SCP Plugin	Jupyter Notebook
User Interface	Graphical User Interface (GUI)	Code-based Interface
Programming Knowledge	Not Required	Required
True Color Composite	Generated	Generated
False Color Composite	Generated	Generated
NDVI Generation	Generated	Generated
Classification Methods	Maximum Likelihood, Random Forest, K-Means	Random Forest
Training Data	ROI Creation using SCP	Imported QGIS Training ROIs
Validation Data	ROI Creation using SCP	Imported QGIS Validation ROIs
Accuracy Assessment	SCP Accuracy Assessment	Scikit-learn Metrics
Overall Accuracy	**92.05%** (Random Forest)	**91.92%** (Random Forest)
Kappa Coefficient	**0.8712**	**0.8595**
Workflow Automation	Limited	High
Reproducibility	Moderate	High
Customization	Limited	Extensive
Model Reusability	Not Available	Supported (.pkl Model)

The comparison demonstrates that both implementations successfully achieved the objectives of land-cover classification using Landsat 9 imagery. The QGIS workflow produced slightly higher classification accuracy, primarily due to the optimized processing pipeline and post-

classification refinement available within the Semi-Automatic Classification Plugin. In contrast, the Python implementation emphasized workflow automation, reproducibility, and flexibility while still achieving a high Overall Accuracy of **91.92%** and a Cohen's Kappa Coefficient of **0.8595**.

While the QGIS workflow is more suitable for rapid analysis through an interactive graphical interface, the Python implementation provides greater flexibility for customization, automation, and integration with machine learning libraries. These characteristics make Python particularly suitable for large-scale geospatial analysis, repetitive processing tasks, and future research involving advanced machine learning techniques.

Advantages of the Python Implementation

The Python-based implementation offers several advantages over the QGIS-based workflow:

- Complete workflow automation through reusable Python scripts.
- High reproducibility, enabling identical results using the same code and input datasets.
- Easy integration with scientific computing and machine learning libraries such as NumPy, Rasterio, GeoPandas, Matplotlib, and Scikit-learn.
- Greater flexibility to modify algorithms, processing parameters, and workflow according to project requirements.
- Ability to save and reuse trained machine learning models (.pkl) for future land-cover prediction.
- Automatic generation of multiple output formats, including GeoTIFF, PNG, CSV, QML, and model files.
- Suitable for batch processing and large-scale satellite image analysis.

Limitations

Despite its advantages, the Python implementation has certain limitations:

- Requires programming knowledge and familiarity with Python libraries.
- Initial workflow development is more time-consuming than GUI-based software.
- Depends on the correct installation and compatibility of external libraries.
- Hyperparameter tuning and model training become computationally expensive for large datasets.
- Classification accuracy is influenced by the quality of training data and parameter selection.
- Additional preprocessing or post-processing techniques may further improve classification performance.

5.15 Chapter Summary

This chapter presented the complete Python-based implementation of the land-cover classification workflow developed for the Jodhpur study area using Landsat 9 multispectral imagery. The implementation complemented the QGIS-based workflow presented in the

previous chapter by providing a programmable, automated, and reproducible approach for satellite image processing and supervised machine learning.

The workflow began with loading and preparing the Landsat 9 multispectral dataset, followed by the generation of the True Color Composite (RGB), False Color Composite (FCC), and Normalized Difference Vegetation Index (NDVI). The training and validation Region of Interest (ROI) datasets prepared during the QGIS implementation were subsequently imported into Python to extract representative spectral samples for supervised learning.

A Random Forest classifier was developed using the extracted training samples, and different model configurations were evaluated to determine an appropriate balance between classification performance and computational efficiency. Based on the experimental analysis, the classifier configured with **300 decision trees** was selected for the final implementation. The trained model was then applied to the complete multispectral image to generate the final land-cover classification raster, which was exported together with visualization outputs, accuracy assessment reports, and the trained machine learning model.

The performance of the developed classifier was evaluated using an independent validation dataset through the computation of the confusion matrix, Overall Accuracy, Cohen's Kappa Coefficient, Producer's Accuracy, User's Accuracy, and the classification report. The Python implementation achieved an **Overall Accuracy of 91.92%** with a **Cohen's Kappa Coefficient of 0.8595**, demonstrating that the Random Forest classifier can effectively distinguish the major land-cover classes present within the study area.

Finally, a comparative analysis between the QGIS and Python implementations highlighted the strengths and limitations of both approaches. While QGIS provided a user-friendly graphical environment for rapid image analysis, Python offered greater flexibility, workflow automation, reproducibility, and integration with machine learning libraries. The ability to save and reuse trained models further enhances the applicability of the Python workflow for large-scale and repetitive geospatial analysis.

Overall, the implementation presented in this chapter successfully achieved the objectives of the study and demonstrated that Python provides an effective platform for satellite image processing and machine learning-based land-cover classification. The developed workflow can be readily extended to other study areas, additional satellite datasets, or advanced classification techniques, making it a valuable framework for future remote sensing and GIS applications.

5.16 References

1. G. Van Rossum and F. L. Drake Jr., *The Python Language Reference*. Python Software Foundation. Available: https://docs.python.org/3/
2. NumPy Developers, *NumPy Documentation*. Available: https://numpy.org/doc/
3. Pandas Development Team, *Pandas Documentation*. Available: https://pandas.pydata.org/docs/
4. Rasterio Developers, *Rasterio Documentation*. Available: https://rasterio.readthedocs.io/
5. GeoPandas Developers, *GeoPandas Documentation*. Available: https://geopandas.org/
6. Matplotlib Development Team, *Matplotlib Documentation*. Available: https://matplotlib.org/stable/

7. Scikit-learn Developers, *Scikit-learn: Machine Learning in Python*. Available: https://scikit-learn.org/stable/
8. L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
9. Joblib Developers, *Joblib Documentation*. Available: https://joblib.readthedocs.io/
10. USGS, *Landsat 9 Data Users Handbook*. Available: https://www.usgs.gov/landsat-missions/landsat-9
11. J. W. Rouse Jr., R. H. Haas, J. A. Schell, and D. W. Deering, "Monitoring Vegetation Systems in the Great Plains with ERTS," *Proceedings of the Third Earth Resources Technology Satellite-1 Symposium*, NASA SP-351, vol. 1, pp. 309–317, 1974.